Práctica 5: Funciones recursivas y programación modular en R

Laboratorio de Matemáticas

Grado en Ingeniería Matemática - Doble Grado en Física e Ingeniería Matemática

Miguel Beltra Vidal

Curso 2026/27

1. Objetivos

El objetivo de esta práctica es aprender el concepto de función recursiva y manejar la programación modular en R. Al finalizarla, el estudiante será capaz de definir funciones recursivas y escribir programas que estén compuestos de varias funciones, cada una de ellas con un propósito único.

2. Funciones recursivas

Una función recursiva es una función que se llama a sí misma para resolver un problema. Siempre tiene dos elementos fundamentales:

- **Caso base**: situación sencilla que puede resolverse directamente y que detiene la recursión.
- **Caso recursivo**: situación en la que la función se vuelve a llamar a sí misma con un problema más pequeño o más sencillo.

Por ejemplo, el factorial de un número natural puede definirse como $0! = 1$ y

$$n! = n \cdot (n-1)! \qquad \text{si } n > 0.$$

Esta definición se puede traducir directamente a una función recursiva:

```
factorial <- function(n) {
  if (n == 0) {
    1
  } else {
    n * factorial(n - 1)
  }
}
```

El caso $n = 0$ representa el caso base, mientras que el caso $n > 0$ es el caso recursivo. Cuando ejecutamos `factorial(4)`, las llamadas se pueden interpretar como:

$$4 \cdot 3! \quad \to \quad 4 \cdot 3 \cdot 2! \quad \to \quad 4 \cdot 3 \cdot 2 \cdot 1! \quad \to \quad 4 \cdot 3 \cdot 2 \cdot 1 \cdot 0! \quad \to \quad 4 \cdot 3 \cdot 2 \cdot 1 \cdot 1.$$

Al final, se alcanza el caso base $0! = 1$ y las llamadas pendientes pueden resolverse:

$$4 \cdot 3 \cdot 2 \cdot 1 \cdot 1 \quad \to \quad 4 \cdot 3 \cdot 2 \cdot 1 \quad \to \quad 4 \cdot 3 \cdot 2 \quad \to \quad 4 \cdot 6 \quad \to \quad 24.$$

La recursividad no es simplemente una forma diferente de escribir un bucle. Es especialmente útil cuando un problema puede expresarse naturalmente en términos de una versión más pequeña del mismo problema.

Ejercicio 1. Define una función recursiva `suma_hasta_rec(n)` que calcule:

$$1 + 2 + 3 + \cdots + n.$$

Por ejemplo:

```
> suma_hasta_rec(5)
```

deberá mostrar:

```
[1] 15
```

En tu script indica cuál es el caso base y cuál el caso recursivo. Calcula las sumas siguientes:

- $\sum_{i=1}^{10} i$.
- $\sum_{i=1}^{20} i$.
- $\sum_{i=1}^{30} i$.

Ejercicio 2. Define una función recursiva `potencia_rec(a,n)` que calcule a^n para $n \geq 0$. Para ello, utiliza la relación recursiva:

$$a^0 = 1, \qquad a^n = a \cdot a^{n-1}.$$

Por ejemplo:

```
> potencia_rec(2,8)
```

deberá mostrar:

```
[1] 256
```

En tu script indica cuál es el caso base y cuál el caso recursivo. Calcula las potencias siguientes:

- 2^{16},
- 3^{14},
- $(-4)^{15}$.

Nota: No utilices el operador `^`.

Ejercicio 3. Define una función recursiva `suma_digitos_rec(n)` que calcule la suma de los dígitos de un número positivo. Utiliza la identidad

$$\texttt{suma_digitos_rec}(n) = (n \text{ mód } 10) + \texttt{suma_digitos_rec}\left(\left\lfloor \frac{n}{10} \right\rfloor\right).$$

Aquí, n mód 10 indica el resto de la división entera, mientras que $\lfloor \frac{n}{10} \rfloor$ representa el cociente de la división. El caso base debe aparecer cuando el número tenga un único dígito. En tu script indica cuál es el caso base y cuál el caso recursivo. Por ejemplo,

```
> suma_digitos_rec(14525)
```

deberá mostrar:

```
[1] 17
```

ya que $1+4+5+2+5=17$. Calcula la suma de los dígitos de los siguientes números:

- 1234567890,
- 122333444455555666666,
- 2^{32} (aquí sí que puedes utilizar el operador `^`).

Ejercicio 4. La sucesión de Fibonacci $\{F_n\}_{n\geq 0}$ se define como: $F_0 = 0$, $F_1 = 1$ y

$$F_n = F_{n-1} + F_{n-2} \qquad \text{si } n \geq 2.$$

Define una función recursiva `fibonacci(n)` que calcule F_n. Comprueba que los primeros términos de la sucesión son:

$$0,\ 1,\ 1,\ 2,\ 3,\ 5,\ 8,\ 13,\ 21,\ 34.$$

En tu script indica cuál es el caso base y cuál el caso recursivo.

Ejercicio 5. **Algoritmo de Euclides**: Define una función recursiva `mcd_rec(a,b)` que calcule el máximo común divisor utilizando la relación de Euclides:

$$\text{mcd}(a, b) = \text{mcd}(b, a \text{ mód } b).$$

Aquí, a mód b indica el resto de la división entera al dividir a entre b. El caso base aparece cuando el segundo argumento es cero. En tu script indica cuál es el caso base y cuál el caso recursivo. Calcula:

- $\text{mcd}(48, 18)$,
- $\text{mcd}(105, 35)$,
- $\text{mcd}(17, 5)$.

Ejercicio 6. **Conjetura de Collatz:** Consideremos la sucesión definida por $a_0 = n$, y

$$a_{k+1} = \begin{cases} a_k/2, & \text{si } a_k \text{ es par,} \\ 3a_k + 1, & \text{si } a_k \text{ es impar.} \end{cases}$$

Por ejemplo, comenzando en $a_0 = 6$, obtenemos:

$$6,\ 3,\ 10,\ 5,\ 16,\ 8,\ 4,\ 2,\ 1.$$

Escribe un programa en R que, dado un número entero positivo `n`, imprima por pantalla los términos de la sucesión hasta alcanzar el valor 1. En tu script indica cuál es el caso base y cuál el caso recursivo. Por ejemplo, para `n = 6` debería obtenerse:

$$6,\ 3,\ 10,\ 5,\ 16,\ 8,\ 4,\ 2,\ 1.$$

Por tanto, se necesitan 8 pasos. A continuación, modifícalo para contar cuántos pasos son necesarios hasta llegar a 1, y haz que sea este el valor de retorno. ¿Cuántos pasos necesitamos en los siguientes casos?

- Empezando en $n = 20$.
- Empezando en $n = 43$.
- Empezando en $n = 1500$.

Nota: Una pregunta surge cuando reflexionamos sobre este ejercicio. ¿Existe algún número n que genera una sucesión que nunca alcanza el 1? Si tienes la tarde libre, trata de demostrar que, independientemente del número del que partamos, la sucesión siempre alcanza el 1, o por el contrario, que existe un número n que genera una sucesión que nunca llega al 1.

3. Programación modular

Una de las ideas fundamentales de la programación es dividir un problema grande en problemas más pequeños. Cada uno de estos problemas puede resolverse mediante una función. Una función puede utilizar otras funciones que hayamos definido anteriormente. De esta forma, podemos construir programas más complejos a partir de piezas sencillas y reutilizables.

Por ejemplo, podemos definir una función que determine si un número es par y utilizarla posteriormente dentro de otra función:

```
es_par <- function(n) {
  n %% 2 == 0
}

siguiente_par <- function(n) {
  if (es_par(n)) {
    n + 2
  } else {
    n + 1
  }
}
```

La función `siguiente_par()` no necesita volver a comprobar qué significa que un número sea par: utiliza la función `es_par()` que ya hemos definido. Algunas de las ideas fundamentales de la programación modular son que **las funciones deben ser sencillas y cumplir un único propósito**, y que **las funciones ayudan a evitar repetir código que ya está escrito**, otorgando una mayor solidez y capacidad de mantenimiento a nuestro código.

Ejercicio 7. Define una función `es_multiplo(a,b)` que devuelva `TRUE` si a es múltiplo de b y `FALSE` en caso contrario. Después, define una función `es_divisor(a,b)` que utilice `es_multiplo()` para determinar si a es divisor de b. Contesta a las siguientes preguntas:

- ¿Cuáles de los siguientes números son múltiplos de 17?

$$255,\ 1156,\ 1397,\ 11645,\ 12617.$$

- ¿Cuáles de los siguientes números son divisores de 1522508?

$$11,\ 19,\ 23,\ 37,\ 52,\ 67,\ 97.$$

Ejercicio 8. Un número primo es un número natural mayor que 1 que solamente es divisible entre 1 y él mismo. Una posible estrategia consiste en recorrer los números desde 2 hasta $n-1$ y comprobar si alguno divide exactamente a n. Por ejemplo, para comprobar si 7 es primo podemos comprobar:

```
> 7 %% 2
> 7 %% 3
> 7 %% 4
> 7 %% 5
> 7 %% 6
```

Si ninguno de estos restos es cero, entonces 7 es primo. Define una función `es_primo(n)` que determine si un número entero positivo es primo. La función deberá devolver `TRUE` si `n` es primo y `FALSE` en caso contrario. Comprueba el resultado para:

$$1,\ 2,\ 3,\ 4,\ 5,\ 10,\ 17,\ 25,\ 29,\ 100.$$

¿Qué tratamiento debe recibir el número 1?

A continuación, define una función `cantidad_primos(a,b)` que cuente cuántos números primos hay entre a y b, ambos incluidos. Por último, crea una función llamada `primos_hasta` que muestre todos los números primos menores o iguales que `n` y muestre la cantidad que tenemos. Por ejemplo:

```
> primos_hasta(20)
```

deberá mostrar:

```
2 3 5 7 11 13 17 19
Total: 8
```

El **Teorema de los Números Primos** establece que la cantidad de números primos menores o iguales que un cierto n es aproximadamente $\frac{n}{\ln(n)}$. Escribe una función llamada `error_teorema_numeros_primos(n)` que, dado un número natural `n`, devuelva el error relativo cometido por el Teorema de los Números Primos, esto es,

$$\text{error} = \frac{|\text{valor exacto} - \text{valor aproximado}|}{\text{valor exacto}}.$$

Finalmente, escribe una función llamada `info_primos(n)` que muestre la siguiente información por pantalla:

```
> info_primos(20)
```

```
2 3 5 7 11 13 17 19
Total: 8
Error relativo del Teorema de los Números Primos: 0.1654795
```

Observa qué pasa con el error relativo cuando $n = 10$, $n = 100$, $n = 1000$ y $n = 10000$.

4. Un último ejercicio

Por supuesto, esto debe estar documentado.

Ejercicio 9. Escribe un fichero `README.md` para tu directorio donde guardarás la práctica 5. El aspecto debe ser similar al de los README que hiciste en la práctica 4 para las prácticas 1 y 3.

5. Entrega y consejos

Sube los ficheros con los ejercicios a tu GitHub en la fecha acordada con el profesor. Para esta práctica:

1. guarda tus scripts en la carpeta de la práctica correspondiente;
2. utiliza nombres de fichero claros y consistentes;
3. comenta el código para facilitar su comprensión;
4. comprueba que puedes ejecutar de nuevo cada script desde el principio;
5. evita depender de instrucciones que hayan quedado solamente en la consola.

Idea clave: un programa complejo puede construirse a partir de funciones pequeñas y reutilizables. Los bucles permiten repetir procesos, los condicionales permiten tomar decisiones y la recursividad permite resolver un problema a partir de una versión más pequeña del mismo problema.